STRUKTURY BAZ DANYCH

PROJEKT 1

Kacper Petla 198186

1. Wprowadzenie

Celem projektu było zaimplementowanie algorytmu sortowania plików sekwencyjnych do czego zastosowano metodę scalania z użyciem wielkich buforów.
Metoda ta polega na początkowym podziale danych wejściowych na runy o wielkości n*b, gdzie:

- n – liczba dostępnych buforów,
- b – liczba rekordów mieszczących się w jednym buforze.

W pierwszej fazie dane z pliku wejściowego są wczytywane partiami, łączone w jeden duży bufor i sortowane (np. algorytmem QuickSort). Tak przygotowane runy są następnie zapisywane do pliku tymczasowego.
W drugiej fazie następuje iteracyjne scalanie - w każdej fazie scala się n - 1 run, wykorzystując n-ty bufor jako bufor wyjściowy. Proces powtarza się, aż do momentu, gdy na taśmie (pliku) pozostanie tylko jedna, w pełni posortowana runa.

2. Implementacja

Program napisano w C++ ,a do implementacji wykorzystano symulację taśm za pomocą plików tekstowych. Program używa 3 taśm jednak w praktyce używane są tylko dwa pliki (wejściowego i wyjściowego), które zamieniają się rolami po każdej fazie sortowania. A trzeci plik ma za zadanie przechowywać nieposortowane rekordy by móc je później porównać

Rekordy mają stałą długość 9 bajtów i zapisane są w formacie 3 duże litery i 6 cyfr (np. AAA123456). W pliku rekordy zapisywane są jeden po drugim bez separatorów. Jeśli liczba rekordów w bloku nie jest całkowita, pozostałe bajty wypełniane są znakami spacji (' ') w celu zachowania integralności struktury blokowej.

Wynik sortowania jest prezentowany w konsoli - każdy rekord wypisywany jest w osobnej linii w takiej kolejności w jakiej są w pliku, co pozwala na łatwą weryfikację poprawności działania programu.

3. Obsługa programu

Po uruchomieniu programu użytkownik definiuje:

- wielkość bufora/bloku (liczbę rekordów w jednym bloku),
- liczbę dostępnych buforów (n).

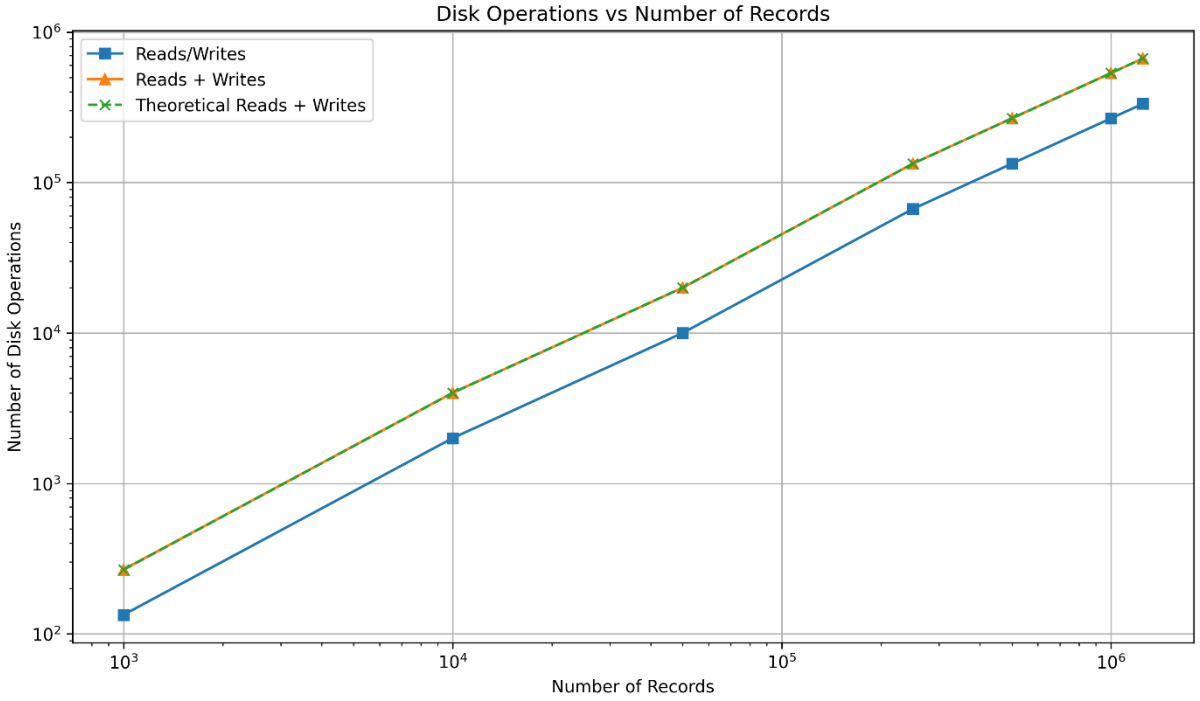
Następnie wyświetlane jest menu, które umożliwia:

- ręczne wprowadzenie rekordów,
- losowe wygenerowanie danych testowych,
- skopiowanie danych z innego pliku,
- uruchomienie procesu sortowania (z opcjonalnym wyświetlaniem rekordów po każdej fazie).

4.Eksperyment i Dane

Eksperyment polegał na generowaniu stopniowo powiększanych zestawów rekordów, sortowaniu ich i zebraniu statystyk wymaganych do analizy (wszystkie w tabeli poniżej).

liczba rekordów	liczba faz	zapisy/ odczyty	zapisy + odczyty	teoretyczna liczba faz	teoretyczne zapisy + odczyty
1000	2	134	267	2	268
10000	3	2001	4000	3	4002
50000	3	10002	20000	3	20004
250000	4	66668	133334	4	133336
500000	4	133336	266667	4	266672
1000000	4	266668	533334	4	533336
1250000	4	333336	666667	4	666672



4.1. Liczba faz sortowania:

Z przeprowadzonego eksperymentu wynika, że liczba faz rośnie wraz ze wzrostem liczby rekordów, ale zależność ta jest stosunkowo łagodna. Dla mniejszych plików (1000 rekordów) wystarczyły 2 fazy, natomiast dla dużych plików (do 1 250 000 rekordów) liczba faz ustabilizowała się na poziomie 4. Jest to zgodne z teorią, która przewiduje liczbę faz jako:

$$\text{fazy} = \lceil \log_n(N/b) \rceil$$

gdzie N to liczba rekordów, b – liczba rekordów w buforze, a n – liczba dostępnych buforów.

Wyniki pokrywają się z wartościami teoretycznymi.

4.2. Liczba odczytów i zapisów stron dyskowych:

Zaobserwowano, że liczba operacji odczytu i zapisu rośnie liniowo wraz z rozmiarem pliku. Dla 10 000 rekordów uzyskano 4000 operacji (suma odczytów i zapisów), dla 1 000 000 rekordów – ponad 530 000 operacji. Porównanie z wartościami teoretycznymi obliczonymi za pomocą wzoru:

$$\text{operacje} = \lceil \text{fazy} * 2(N/b) \rceil$$

gdzie N to liczba rekordów, b – liczba rekordów w buforze.

Pokazuje, że rzeczywista liczba odczytów i zapisów bardzo dobrze zgadza się z obliczeniami teoretycznymi Odchylenia wynikają głównie z zaokrągleń i ewentualnego wypełniania ostatniego bloku spacjami.

4.3. Efektywność algorytmu:

Algorytm z użyciem wielkich buforów okazał się efektywny, ponieważ liczba faz rośnie powoli, a każda faza przetwarza wiele rekordów w pamięci, minimalizując liczbę operacji dyskowych. W praktyce oznacza to, że nawet bardzo duże pliki (ponad milion rekordów) można posortować przy ograniczonej liczbie buforów w pamięci operacyjnej.

4.4. Porównanie wyników praktycznych z teoretycznymi:

Różnice pomiędzy wartościami praktycznymi a teoretycznymi są minimalne i wynikają głównie z:

- konieczności wypełnienia niepełnych bloków spacjami (co zwiększa liczbę zapisów),
- ograniczeń zaokrągleń w obliczeniach liczby faz i bloków.

Ogólnie, wyniki eksperymentu potwierdzają poprawność implementacji algorytmu i zgodność z przewidywaniami teoretycznymi.

5. Podsumowanie:

Przeprowadzony eksperyment potwierdził, że algorytm sortowania przez scalanie z użyciem wielkich buforów jest efektywny dla dużych plików sekwencyjnych. Liczba faz pozostaje ograniczona, a liczba operacji dyskowych rośnie liniowo wraz ze wzrostem liczby rekordów. Dzięki temu metoda ta jest skalowalna i praktyczna do sortowania dużych zbiorów danych przy ograniczonej ilości pamięci.